

SORBONNE UNIVERSITÉ

UE PC3R — Programmation Concurrente, Répartie, Réactive et réticulaire

Fantasy MMA

Rapport de projet

Application web de pronostics sur les combats UFC

Auteur Ramy SAIL

Numéro étudiant 21501974

Année universitaire 2025 / 2026

Table des matières

1	Introduction	2
2	Analyse fonctionnelle du projet	2
2.1	Cas d'utilisation principaux	2
2.2	Scénario d'utilisation	3
2.3	Fonctionnalités attendues	3
3	Architecture générale	3
3.1	Vue d'ensemble du système	3
3.2	Plan des écrans et gabarits de pages	4
4	Modélisation des données	4
4.1	Pourquoi commencer par la base de données ?	4
4.2	Schéma relationnel	4
4.3	Choix de conception notables	6
5	Serveur Go	6
5.1	Connexion à la base et organisation du code	6
5.2	Alimentation de la base via l'API ESPN	6
5.3	Modélisation des tâches concurrentes	8
5.4	Logique métier côté serveur	8
5.4.1	Pronostics et fenêtre de prédiction	8
5.4.2	Attribution des points	8
5.4.3	Cohérence des cartes annulées	9
5.5	Authentification et sessions	9
6	Client React	9
6.1	Choix technologiques	9
6.2	Création et traitement des requêtes	9
7	API REST et échanges client–serveur	10
7.1	Ressources principales	10
7.2	Exemples d'invocation	10
7.3	Description des échanges	13
7.4	Gestion du CORS	13
8	Déploiement	13
9	Conclusion	14

1 Introduction

Fantasy MMA est une application web qui permet aux utilisateurs de pronostiquer les résultats des combats d'arts martiaux mixtes (MMA), et plus précisément des cartes de l'UFC. L'application agrège automatiquement les données sportives depuis une source externe, expose ces données à travers une API REST, et offre à chaque utilisateur la possibilité d'enregistrer ses prédictions, de suivre les points obtenus une fois les combats terminés, et de se comparer aux autres participants via un classement global.

Le projet mobilise une architecture complète en quatre couches :

- un **client React** qui structure l'interface, gère le routage et pilote les requêtes AJAX vers le serveur.
- un **serveur Go** qui expose une API REST orientée ressource, gère l'authentification par session, orchestre l'import des données et applique les règles métier (fermeture des prédictions, attribution des points).
- une **base PostgreSQL** qui persiste les cartes, les combats, les combattants, les utilisateurs, les prédictions et les sessions.
- une **API externe (ESPN, endpoint MMA)** qui alimente le système en données de cartes, de combats et de résultats.

2 Analyse fonctionnelle du projet

2.1 Cas d'utilisation principaux

Les scénarios suivants reprennent le cœur fonctionnel de l'application.

UC1 : Consultation publique des cartes.

Un visiteur non authentifié peut consulter la liste des cartes UFC, le détail d'une carte (lieu, date, statut) et la liste des combats associés avec les combattants, leurs records et leurs visuels. Aucune création de compte n'est nécessaire.

UC2 : Création d'un compte et ouverture des fonctionnalités avancées.

Lorsqu'un visiteur s'inscrit ou se connecte, son état change : il peut désormais enregistrer des prédictions et consulter son historique personnel de pronostics.

UC3 : Pronostic sur un combat.

Un utilisateur connecté sélectionne une carte, consulte les combats, puis désigne pour chaque combat le combattant qu'il pense vainqueur. La prédiction n'est acceptée que si les prédictions de la carte ne sont pas encore fermées (la fermeture intervient un jour avant la carte) et si le combattant choisi participe réellement au combat.

UC4 : Mise à jour différée des scores.

Après la tenue d'une carte, le serveur synchronise les résultats avec ESPN. Pour chaque combat terminé, les prédictions correctes se voient attribuer les points associés au combat. Les prédictions portant sur un combat annulé sont supprimées.

UC5 : Consultation du classement global.

Tout utilisateur, connecté ou non, peut consulter le classement global trié sur le total de points, puis sur le nombre de bonnes prédictions, puis sur le nombre total de prédictions.

2.2 Scénario d'utilisation

Un utilisateur A consulte librement les cartes à venir. Un utilisateur B, connecté, ouvre une carte UFC à venir et pronostique le vainqueur de chaque combat avant la date de fermeture. Les deux quittent ensuite l'application. Une fois la carte disputée et les résultats synchronisés par le serveur, ils reviennent : B obtient les points correspondant à ses bons pronostics et progresse au classement, tandis que les utilisateurs n'ayant pas prédit ou ayant mal prédit ne gagnent rien.

2.3 Fonctionnalités attendues

Acteur	Fonctionnalités
Visiteur	Consulter les cartes, parcourir les combats à venir et passés, afficher le détail d'une carte et de ses combats, consulter le classement global.
Utilisateur connecté	S'inscrire, se connecter, se déconnecter, enregistrer ou modifier un pronostic, consulter l'historique détaillé de ses prédictions et les points obtenus.
Serveur	Synchroniser les données sportives, protéger les actions privées par session, fermer les prédictions à l'approche d'une carte, attribuer les points après les combats, traiter les combats annulés.
Base de données	Assurer la persistance des entités métier, garantir l'unicité d'une prédiction par couple (utilisateur, combat), indexer les jetons de session.

3 Architecture générale

3.1 Vue d'ensemble du système

L'architecture distingue clairement ce qui est affiché côté client, ce qui est calculé côté serveur et ce qui est persisté en base.

La figure 1 illustre les quatre composants du système et les flux entre eux.

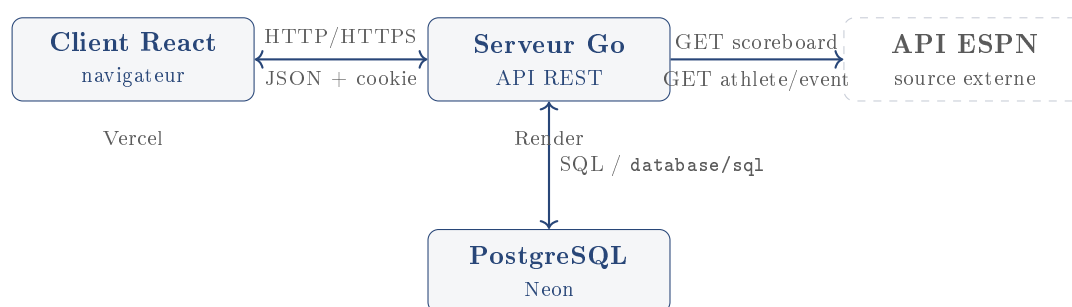


Figure 1: Architecture globale de Fantasy MMA. Le serveur Go est le seul composant à interroger l'API ESPN ; le client ne communique qu'avec le serveur Go.

Cette architecture a deux propriétés importantes :

- le **client** ne contient pas de logique métier sensible : il affiche les données, transforme les réponses JSON et guide les interactions.
- le **serveur** concentre les décisions critiques : authentification, validation des données, fermeture des prédictions, attribution des points, synchronisation avec l'API externe et

accès à la base de données.

3.2 Plan des écrans et gabarits de pages

En React, ces modèles correspondent directement aux composants de page déclarés dans `App.jsx`.

Écran	Composant	Rôle
Accueil	HomePage	Présentation de l'application et accès rapide à la prochaine carte ainsi qu'à quelques statistiques globales.
Liste des cartes	CardsPage	Liste de toutes les cartes triées par date.
Détail d'une carte	CardDetailsPage	Affiche le lieu, la date, le statut, les combats et le formulaire de pronostic.
Classement	LeaderboardPage	Affiche le classement global des utilisateurs.
Mes prédictions	MyPredictionsPage	Historique des pronostics de l'utilisateur connecté (route protégée).
Connexion inscription	/ LoginPage, RegisterPage	Formulaires d'entrée dans l'espace authentifié.

Les routes privées sont protégées côté client par un composant `ProtectedRoute`, et côté serveur par la vérification systématique du cookie de session.

4 Modélisation des données

4.1 Pourquoi commencer par la base de données ?

Avant même de détailler le backend et le frontend, il fallait identifier les objets métier manipulés par le système : **cartes**, **combats**, **combattants**, **utilisateurs**, **prédictions** et **sessions**. Cette étape conditionne le reste : une API REST bien structurée et une interface lisible supposent une base cohérente.

Le projet repose sur PostgreSQL pour deux raisons :

- la simplicité d'intégration avec le paquet standard Go `database/sql`.
- mon expérience avec ce type de base relationnelle dans de précédentes UE et projets universitaires.

4.2 Schéma relationnel

Le modèle s'organise autour d'un noyau sportif (cartes, combattants, combats), d'une logique de jeu (prédictions des utilisateurs) et d'une logique d'authentification (utilisateurs, sessions).

La figure 2 présente le modèle entité-association du projet.

Les tables principales sont les suivantes :

- **cards**, **fighters** et **fights** forment le noyau sportif. Chaque entité porte un `external_id` unique correspondant à l'identifiant ESPN, ce qui permet des insertions idempotentes (*upsert*) lors des synchronisations.

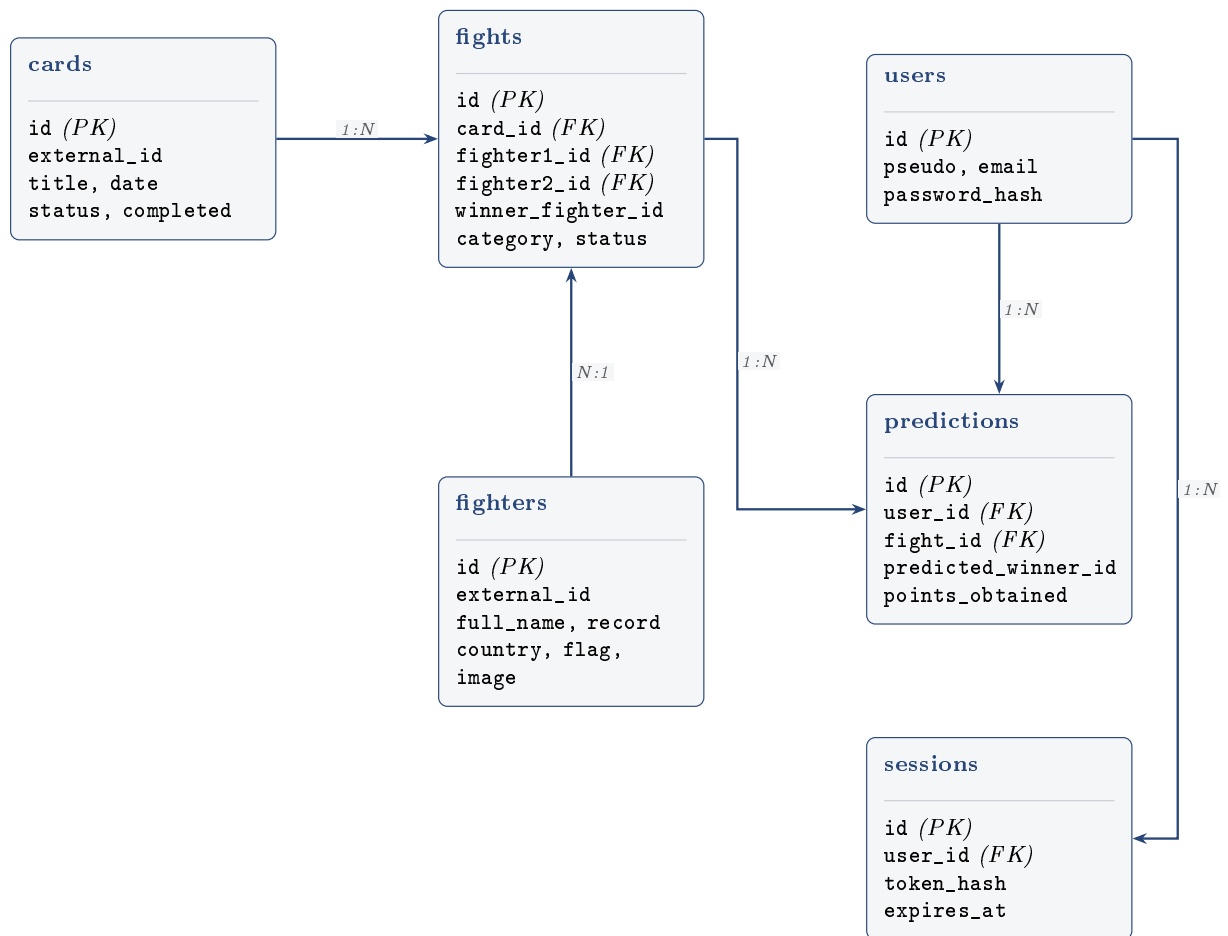


Figure 2: Modèle entité-association de Fantasy MMA. Les flèches indiquent le sens des clés étrangères. La contrainte `UNIQUE(user_id, fight_id)` sur `predictions` garantit une seule prédiction par combat et par utilisateur.

- **users** et **predictions** portent la logique de jeu. La table **predictions** stocke le combattant prédit et les points obtenus, avec une contrainte **UNIQUE (user_id, fight_id)** qui garantit une seule prédiction par combat et par utilisateur.
- **sessions** porte la logique d'authentification : elle associe un **token_hash** à un utilisateur et à une date d'expiration.

4.3 Choix de conception notables

Identifiants externes et upsert. Plutôt que de réinsérer les cartes et combats à chaque synchronisation, les requêtes utilisent la clause **INSERT ... ON CONFLICT (external_id) DO UPDATE**. Cela rend la synchronisation idempotente : une carte déjà connue est mise à jour (statut, résultat) sans créer de doublon.

Points par combat. Le nombre de points attribués pour une bonne prédiction est stocké au niveau de chaque combat (**points_good_prediction**). Ce choix laisse la porte ouverte à une pondération future (par exemple valoriser davantage le combat principal), même si la valeur est actuellement fixée par défaut.

Suppression en cascade. Les contraintes **ON DELETE CASCADE** sur les prédictions et les sessions garantissent qu'aucune ligne orpheline ne subsiste si un utilisateur, une carte ou un combat est supprimé.

5 Serveur Go

5.1 Connexion à la base et organisation du code

Le backend Go est organisé en plusieurs paquets aux responsabilités distinctes :

- **database** : connexion PostgreSQL (**db.go**), types métier (**types.go**), requêtes SQL (**queries.go**) et fonctions de scan (**utils.go**).
- **espn** : intégration à l'API externe – client HTTP (**client.go**), types de désérialisation (**espn_types.go**), logique de synchronisation (**sync.go**) et utilitaires (**utils.go**).
- **handlers** : les gestionnaires HTTP regroupés par domaine (**auth.go**, **cards.go**, **predictions.go**, **ranking.go**), les types de requête/réponse (**types.go**) et des utilitaires partagés (**utils.go**).
- **server** : déclaration du multiplexeur, du middleware CORS et démarrage (**server.go**).
- **main.go** : démarrage du serveur et lancement des goroutines de synchronisation et de nettoyage.

Go n'offre pas nativement d'ORM intégré. le projet utilise donc directement **database/sql** avec le pilote **lib/pq**. La fonction **InitDB()** construit la chaîne de connexion à partir de la variable d'environnement **DATABASE_URL** (ou d'une valeur locale par défaut) et vérifie la connexion via **db.Ping()**.

5.2 Alimentation de la base via l'API ESPN

L'application s'appuie sur deux domaines distincts de l'API JSON publique d'ESPN pour le MMA UFC. Aucune clé d'API n'est requise. Le paquet **espn** (fichier **client.go**) déclare les

deux bases d'URL et expose quatre fonctions de récupération correspondant à quatre endpoints complémentaires.

Endpoint 1 — Scoreboard général (calendrier de saison)

```
GET https://site.api.espn.com/apis/site/v2/sports/mma/ufc/scoreboard
```

Utilisé par `fetch()` lors de `InitialSync` et `UpdateSync`. La réponse contient le champ `leagues[0].calendar`, un tableau d'entrées dont chacune décrit une carte à venir avec sa `startDate` et un lien `$ref` vers l'événement. Ce calendrier sert à itérer sur toutes les dates de la saison.

Endpoint 2 — Scoreboard par date (détail d'une carte)

```
GET https://site.api.espn.com/apis/site/v2/sports/mma/ufc/scoreboard?dates=YYYYMMDD
```

Utilisé par `fetchByDate()`, appelé indirectement via `fetchRightDate()`. Pour chaque date du calendrier, cette URL renvoie le champ `events[]` avec le détail complet de la carte : métadonnées de l'événement (`id`, `name`, `date`, `status`), lieu (`venues[]`), et surtout la liste des combats (`competitions[]`) avec les deux concurrents. C'est la source principale pour peupler les tables `cards` et `fighters`.

Tolérance aux décalages de date. La même carte peut être retournée par l'API sous la date exacte, la veille ou le lendemain selon le fuseau horaire du lieu. La fonction `fetchRightDate` enchaîne jusqu'à trois appels (date exacte, J-1, J+1) et conserve la première réponse non vide. La fonction `GetDayDelta` gère par ailleurs le *parsing* des différents formats de dates produits par l'API (2006-01-02T15:04Z, 2006-01-02T15:04:05Z, etc.).

Endpoint 3 — Détail d'un événement (résultats officiels)

```
GET https://sports.core.api.espn.com/v2/sports/mma/leagues/ufc/events/{id}
```

Utilisé par `fetchEvent()`, appelé dans `syncPredictions`. Cet endpoint du *core* ESPN renvoie un objet `ESPNEvent` complet pour un événement connu par son identifiant externe. Une fois la carte terminée (`event.status.type.completed == true`), cet appel fournit les champs `competitors[k].winner` qui permettent d'identifier le vainqueur officiel de chaque combat et de mettre à jour les prédictions en base.

Endpoint 4 — Détail d'un athlète (informations combattant)

```
GET https://sports.core.api.espn.com/v2/sports/mma/leagues/ufc/athletes/{id}
```

Utilisé par `fetchAthlete()`, appelé pour chaque compétiteur de chaque combat lors de la synchronisation. Cet appel récupère les données enrichies du combattant qui ne figurent pas dans le scoreboard : nom complet, drapeau du pays (`flag.href` / `flag.alt`), et photo (`headshot.href`). Ces champs alimentent la table `fighters`. Si l'un des deux combattants porte le nom TBA (*to be assigned*), le combat entier est ignoré jusqu'à la prochaine synchronisation.

Récapitulatif des appels ESPN. Le tableau suivant résume les quatre endpoints et leur rôle dans le pipeline :

Fonction Go	Rôle	Appelée par
<code>fetch()</code>	Calendrier de saison	<code>InitialSync</code> , <code>UpdateSync</code>
<code>fetchByDate(date)</code>	Détail carte + combats	<code>InitialSync</code> , <code>UpdateSync</code>
<code>fetchEvent(id)</code>	Résultats officiels	<code>syncPredictions</code>
<code>fetchAthlete(id)</code>	Profil combattant	<code>syncFight</code> (×2 par combat)

5.3 Modélisation des tâches concurrentes

Le serveur exploite directement les primitives de concurrence de Go. Au démarrage, `main.go` lance trois goroutines indépendantes :

1. une synchronisation initiale (`InitialSync`) qui remplit une base vierge avec toutes les cartes de la saison.
2. une synchronisation périodique (`SyncTicker`) qui se déclenche à une heure fixe de la journée puis se répète à intervalle régulier, à l'aide d'un `time.Ticker`.
3. un nettoyage horaire des sessions expirées, également cadencé par un `time.Ticker`.

On obtient ainsi un modèle simple et lisible : les synchronisations s'exécutent en arrière-plan sans bloquer le service HTTP, qui reste disponible pour répondre aux clients pendant les imports.

5.4 Logique métier côté serveur

5.4.1 Pronostics et fenêtre de prédiction

Lorsqu'un utilisateur soumet un pronostic via `POST /predictions`, le serveur effectue une série de vérifications avant tout enregistrement :

1. il vérifie que l'utilisateur est authentifié (présence et validité du cookie de session).
2. il lit en une requête le statut du combat et de la carte associée, ainsi que la date de la carte.
3. il refuse la prédiction si les prédictions sont fermées, c'est-à-dire si la date courante dépasse la date de la carte moins le délai de fermeture (`closePredictionsDeadline`, fixé à un jour).
4. il refuse la prédiction si le combat ou la carte est déjà terminé.
5. il refuse la prédiction si le combattant choisi ne participe pas au combat.

Si toutes les vérifications passent, la prédiction est enregistrée par un `upsert` : une nouvelle prédiction est créée, ou la prédiction existante pour ce couple (utilisateur, combat) est mise à jour. La fermeture des prédictions avant la tenue de la carte évite par construction toute condition de course avec la mise à jour des résultats.

5.4.2 Attribution des points

Le recalcul des points ne dépend pas du frontend. Lors d'une synchronisation, dès qu'une carte est marquée terminée par ESPN, le serveur parcourt ses combats :

- si un combat est annulé (statut `STATUS_CANCELED`), les prédictions associées sont supprimées.
- sinon, le vainqueur officiel est identifié à partir du champ `winner` renvoyé par l'API, et les prédictions correctes se voient attribuer les points du combat via une seule requête `UPDATE` jointe sur la table des combattants.

Cette règle reste volontairement simple afin de rendre le système transparent pour l'utilisateur.

5.4.3 Cohérence des cartes annulées

L'API ESPN ne fournit pas toujours explicitement un statut `STATUS_CANCELED` : un combat annulé peut rester marqué `STATUS_SCHEDULED` pendant plusieurs jours. La fonction `assureCardCoherent` corrige ce comportement : si une carte est terminée alors que l'un de ses combats ne l'est pas, ce combat est requalifié comme annulé en base. Cela garantit la cohérence avant l'étape d'attribution des points.

5.5 Authentification et sessions

L'authentification repose sur des sessions opaques côté serveur plutôt que sur des jetons auto-portés.

À l'inscription ou à la connexion, le serveur génère un jeton aléatoire de 32 octets encodé en base64 URL-safe. Seul le *haché* SHA-256 de ce jeton est stocké en base, dans la table `sessions`, avec une date d'expiration fixée à sept jours. Le jeton en clair est renvoyé au client dans un cookie `session_token` marqué `HttpOnly`, `Secure` et `SameSite=None` (nécessaire pour le partage de cookies entre domaines distincts en production).

À chaque requête protégée, le serveur calcule le haché du jeton reçu et recherche en base une session non expirée correspondante pour retrouver l'identifiant utilisateur. Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec `bcrypt`.

6 Client React

6.1 Choix technologiques

Le frontend a été réalisé en **React** avec **Vite**, et stylé avec **Tailwind CSS**. La structure du client suit un schéma simple :

- `App.jsx` définit le squelette global et les routes d'écran via `react-router-dom`.
- `AuthContext.jsx` centralise l'état d'authentification et expose les actions `login`, `register` et `logout`.
- le dossier `services/` encapsule tous les appels HTTP, regroupés par domaine (`authService`, `cardService`, `leaderboardService`, `predictionService`).
- les composants de pageinstancient les gabarits décrits plus haut.

6.2 Création et traitement des requêtes

Toutes les interactions réseau passent par une fonction centralisée `apiRequest` (fichier `api.js`). Celle-ci construit l'URL à partir du préfixe configurable, ajoute les en-têtes nécessaires, sérialise automatiquement les corps JSON et active le mode `credentials: "include"`. Ce dernier point est essentiel, car l'authentification repose sur le cookie de session `HttpOnly` posé par le serveur.

La fonction normalise également la gestion des erreurs : les réponses non **2xx** sont converties en une exception **ApiError** portant le statut HTTP et le message d'erreur renvoyé par le serveur, ce qui permet aux composants d'afficher un retour utilisateur cohérent.

Contexte d'authentification. Au montage de l'application, **AuthProvider** appelle `/auth/me` pour déterminer si une session valide existe déjà. L'état utilisateur ainsi obtenu est partagé à tout l'arbre React via un contexte, ce qui évite de redemander l'identité à chaque page et permet aux routes protégées de réagir immédiatement.

7 API REST et échanges client–serveur

7.1 Ressources principales

L'API est organisée par ressource. Les routes de lecture publiques ne nécessitent pas d'authentification. Les routes d'écriture liées à un utilisateur l'exigent.

Méthode	Route	Rôle	Auth.
GET	<code>/cards</code>	Liste de toutes les cartes.	Non
GET	<code>/cards/{id}</code>	Détail d'une carte (+ état des prédictions).	Non
GET	<code>/cards/{id}/fights</code>	Combats d'une carte avec combattants.	Non
GET	<code>/cards/{id}/predictions/me</code>	Prédictions de l'utilisateur pour une carte.	Oui
GET	<code>/ranking</code>	Classement global des utilisateurs.	Non
POST	<code>/auth/register</code>	Crée un compte et ouvre une session.	Non
POST	<code>/auth/login</code>	Ouvre une session et pose le cookie.	Non
POST	<code>/auth/logout</code>	Ferme la session courante.	Oui
GET	<code>/auth/me</code>	Renvoie l'utilisateur courant.	Oui
POST	<code>/predictions</code>	Enregistre ou met à jour un pronostic.	Oui
POST	<code>/predictions/bulk</code>	Enregistre plusieurs pronostics en une seule requête (validation atomique).	Oui
GET	<code>/predictions/me</code>	Historique des prédictions de l'utilisateur.	Oui

7.2 Exemples d'invocation

Chaque exemple montre la requête HTTP et un extrait représentatif de la réponse JSON.

1) Lister toutes les cartes

GET `/cards`

HTTP/1.1 200 OK

```
[
  {
    "id": 12,
    "external_id": "600058807",
    "title": "UFC Fight Night: Della Maddalena vs. Prates",
    "date": "2026-05-02 08:00:00",
    "status": "STATUS_FINAL",
    "completed": true,
    "venue_name": "RAC Arena (AUS)",
    "city": "Perth",
    "region": "WA",
    "country": "Australia"
  },
  {
```

```
"id": 13,
"external_id": "600060012",
"title": "UFC 315: Muhammad vs. Della Maddalena",
"date": "2026-05-10 23:00:00",
"status": "STATUS_SCHEDULED",
"completed": false,
"venue_name": "Bell Centre",
"city": "Montreal",
"region": "QC",
"country": "Canada"
}
]
```

2) Récupérer les combats d'une carte

GET /cards/13/fights

HTTP/1.1 200 OK

```
[
{
  "id": 101,
  "external_id": "401863200",
  "category": "Welterweight",
  "status": "STATUS_SCHEDULED",
  "completed": false,
  "points_good_prediction": 10,
  "fighter1": {
    "id": 42,
    "full_name": "Belal Muhammad",
    "record": "24-3-0",
    "fighter_country_name": "USA",
    "fighter_country_flag_url": "https://a.espncdn.com/.../usa.png",
    "fighter_image_url": "https://a.espncdn.com/.../muhammad.png"
  },
  "fighter2": {
    "id": 43,
    "full_name": "Jack Della Maddalena",
    "record": "17-2-0",
    "fighter_country_name": "Australia",
    "fighter_country_flag_url": "https://a.espncdn.com/.../aus.png",
    "fighter_image_url": "https://a.espncdn.com/.../maddalena.png"
  },
  "winner_id": { "Int64": 0, "Valid": false }
}
]
```

3) Enregistrer plusieurs pronostics en une seule requête (bulk)

POST /predictions/bulk

Content-Type: application/json

```
{
  "predictions": [
    { "fight_id": 101, "predicted_winner_id": 42 },
    { "fight_id": 102, "predicted_winner_id": 55 },
    { "fight_id": 103, "predicted_winner_id": 61 }
  ]
}
```

```
HTTP/1.1 200 OK
{ "saved_count": 3 }
```

Toutes les prédictions de la liste sont validées *avant* toute écriture : si l'une est invalide (combat terminé, prédictions fermées, combattant absent), l'intégralité de la requête est rejetée avec le code d'erreur correspondant.

4) Consulter l'historique de ses prédictions

GET /predictions/me

Cookie: session_token=<token>

```
HTTP/1.1 200 OK
[
  {
    "id": 12,
    "card_title": "UFC Fight Night: Della Maddalena vs. Prates",
    "card_date": "2026-05-02 08:00:00",
    "status": "completed",
    "total_points": 20,
    "good_predictions": 2,
    "total_predictions": 3,
    "possible_points": 30,
    "fights": [
      {
        "fight_id": 88,
        "category": "Welterweight",
        "fighter1": "Jack Della Maddalena",
        "fighter2": "Carlos Prates",
        "predicted_winner": "Jack Della Maddalena",
        "official_winner": "Jack Della Maddalena",
        "points_obtained": 10,
        "points_good_prediction": 10,
        "status": "correct"
      },
      {
        "fight_id": 89,
        "category": "Heavyweight",
        "fighter1": "Sergei Pavlovich",
        "fighter2": "Jailton Almeida",
        "predicted_winner": "Sergei Pavlovich",
        "official_winner": "Jailton Almeida",
        "points_obtained": 0,
        "points_good_prediction": 10,
        "status": "wrong"
      }
    ]
  }
]
```

5) Consulter le classement global

GET /ranking

```
HTTP/1.1 200 OK
[
  {
    "rank": 1,
    "pseudo": "alice",
```

```
"total_points": 80,
"good_predictions": 8,
"total_predictions": 12
},
{
  "rank": 2,
  "pseudo": "bob",
  "total_points": 70,
  "good_predictions": 7,
  "total_predictions": 10
}
]
```

7.3 Description des échanges

La séquence la plus représentative est celle du détail d'une carte :

1. le client demande la carte via `GET /cards/{id}`, qui renvoie aussi un indicateur `predictions_closed` et la date limite de pronostic calculés côté serveur.
2. il enchaîne avec `GET /cards/{id}/fights` pour obtenir la liste des combats et des combattants.
3. si l'utilisateur est connecté, il charge `GET /cards/{id}/predictions/me` pour présélectionner les pronostics déjà saisis.
4. l'utilisateur soumet ses choix en une seule requête groupée via `POST /predictions/bulk` : le serveur valide atomiquement toutes les prédictions de la carte avant d'effectuer les écritures en base.

Cette organisation évite de produire une réponse géante côté backend : chaque ressource conserve une responsabilité propre, ce qui améliore la lisibilité de l'API et la maintenance du code.

7.4 Gestion du CORS

Le client et le serveur étant hébergés sur des domaines distincts en production, un middleware CORS (`corsMiddleware` dans `server.go`) autorise explicitement l'origine de la requête, active `Access-Control-Allow-Credentials` pour permettre l'envoi du cookie de session, et répond aux requêtes de pré-vérification `OPTIONS`. La combinaison `SameSite=None` + `Secure` sur le cookie est indispensable pour que le navigateur accepte de le transmettre entre les deux domaines.

8 Déploiement

L'application Fantasy MMA a été conçue pour être intégralement déployée dans le cloud tout en respectant la séparation stricte des couches. Le choix de l'infrastructure s'est articulé autour d'une contrainte claire : mettre le projet en ligne en exploitant au mieux les offres gratuites (*free tiers*).

La répartition des services est la suivante :

- **Vercel** pour le client React : adapté à l'hébergement de *Single Page Applications*, avec des déploiements continus à chaque mise à jour du dépôt.

- **Render** pour le serveur Go : son offre gratuite permet d'exécuter le backend. L'instance peut se mettre en veille après une période d'inactivité, ce qui explique la latence du premier chargement, mais reste acceptable pour un projet universitaire.
- **Neon** pour la base PostgreSQL : cette solution *serverless* propose une offre gratuite et s'intègre naturellement avec la bibliothèque `database/sql` de Go via la variable `DATABASE_URL`.

Le déploiement a nécessité quelques adaptations du code. D'une part, la configuration du backend est entièrement pilotée par des variables d'environnement (`DATABASE_URL`, `PORT`), ce qui correspond au mode de configuration attendu par Render. D'autre part, l'hébergement séparé du client et du serveur a imposé une gestion soignée du CORS et des cookies de session : le cookie est posé avec `SameSite=None` et `Secure`, et le middleware serveur reflète l'origine appelante tout en autorisant les *credentials*.

9 Conclusion

Fantasy MMA est une application web complète qui combine ingestion de données sportives externes, persistance relationnelle, API REST, authentification par session et interface utilisateur moderne.

Le projet est particulièrement intéressant d'un point de vue PC3R car il oblige à raisonner sur toute la chaîne : le visiteur, l'utilisateur connecté, la ressource REST, le stockage, les échanges JSON, la concurrence des tâches de synchronisation et les contraintes de déploiement multi-domaines.